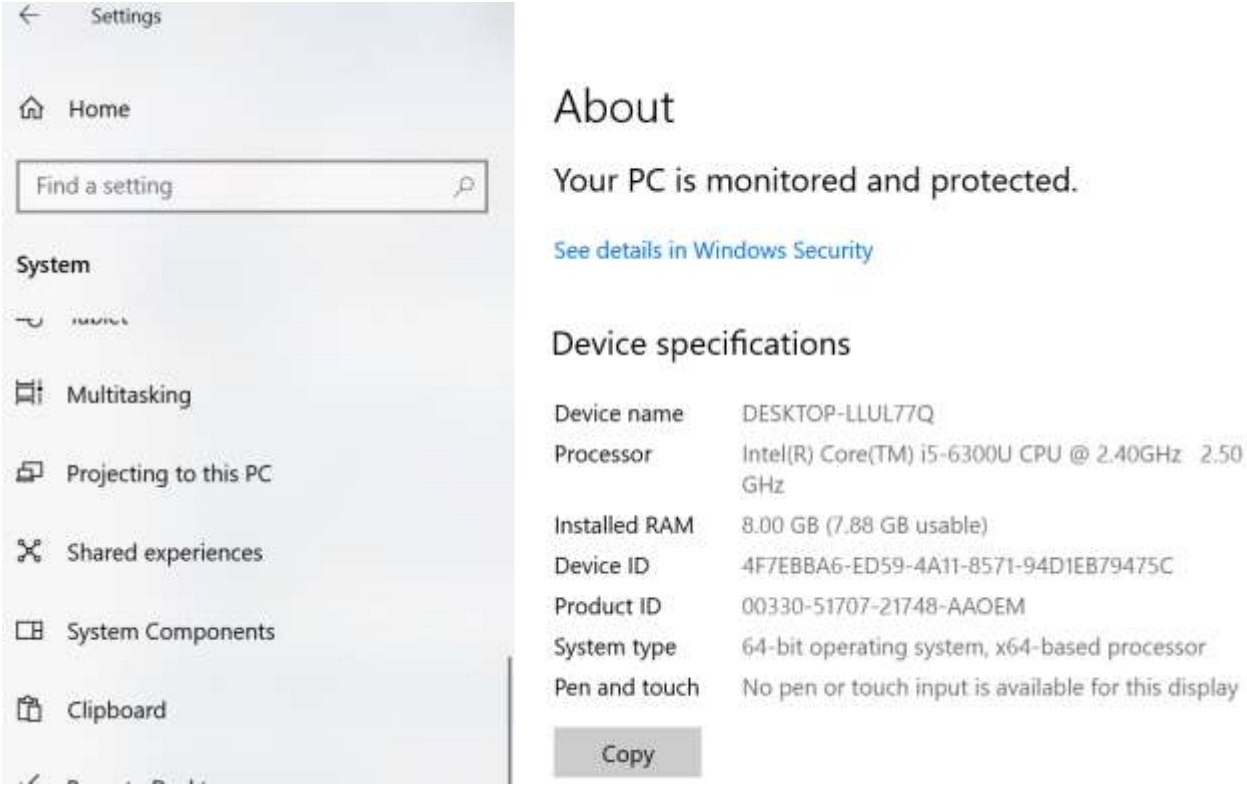
Installing GIT

On Windows and Mac

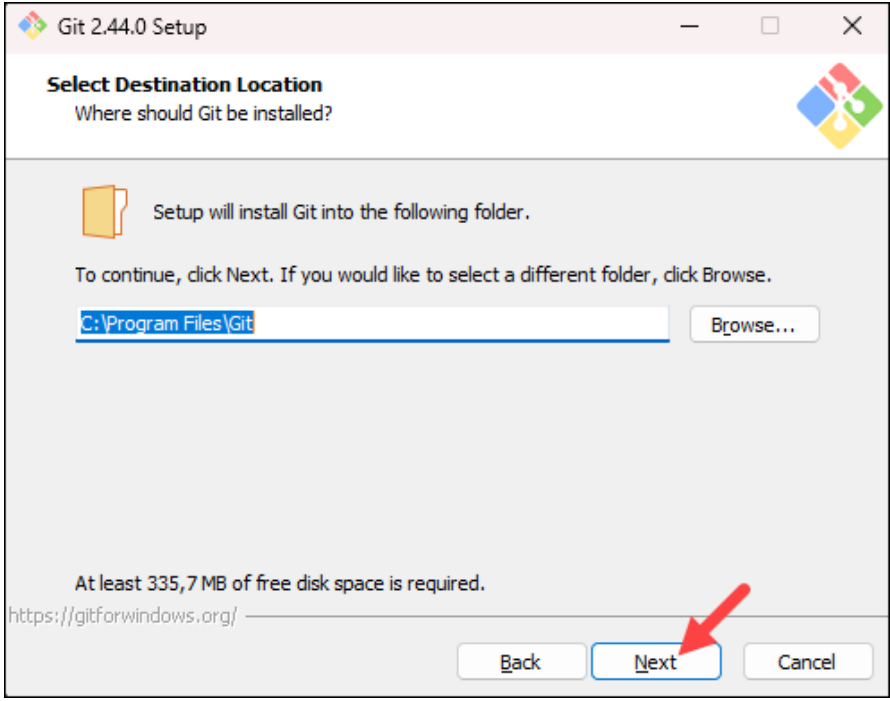
GIT Installation for Windows

System settings check for choice between 64/32 bit versions

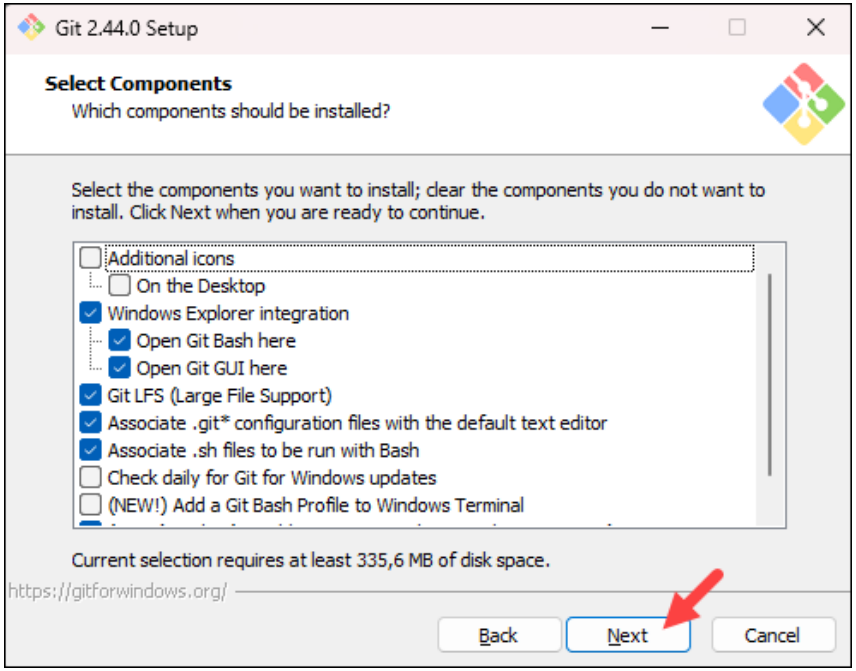
- If system is 32 bit, a 64 bitsoftware wouldn't work as desired
- Go to settings-> system->about->system type to verify system architecture

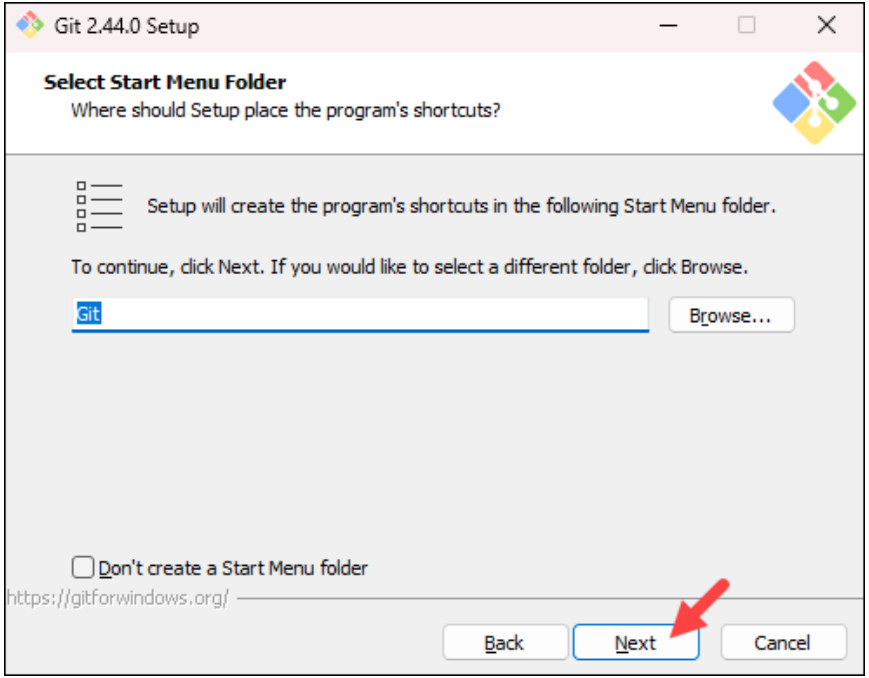


Review license and installation path



Installation Contd





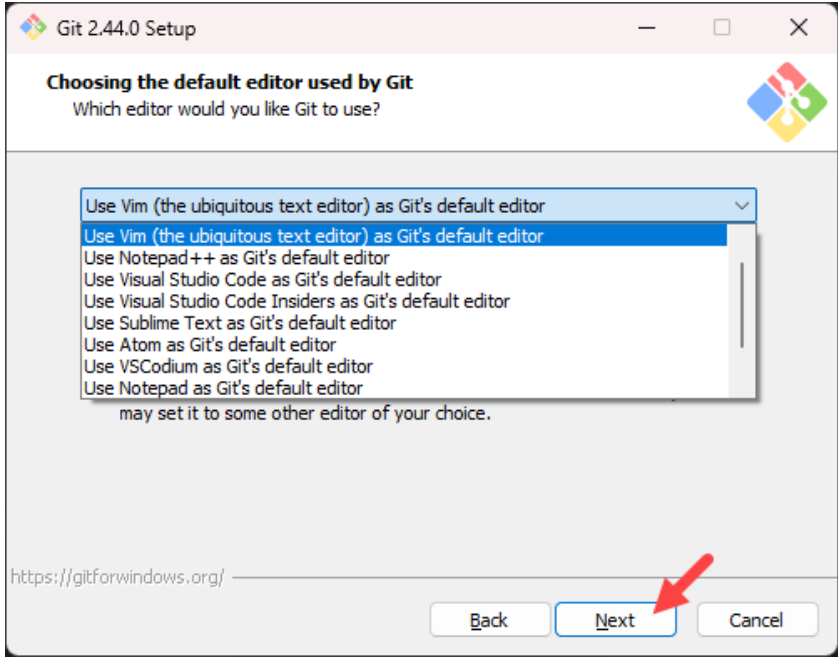
Choosing Text Editor

You need a text editor with Git because it's used for various tasks, including: writing commit messages, resolving conflicts, and editing configuration files

Select a text editor you want to use with Git. Use the drop-down menu to select Notepad++ (or whichever text editor you prefer) and click Next.

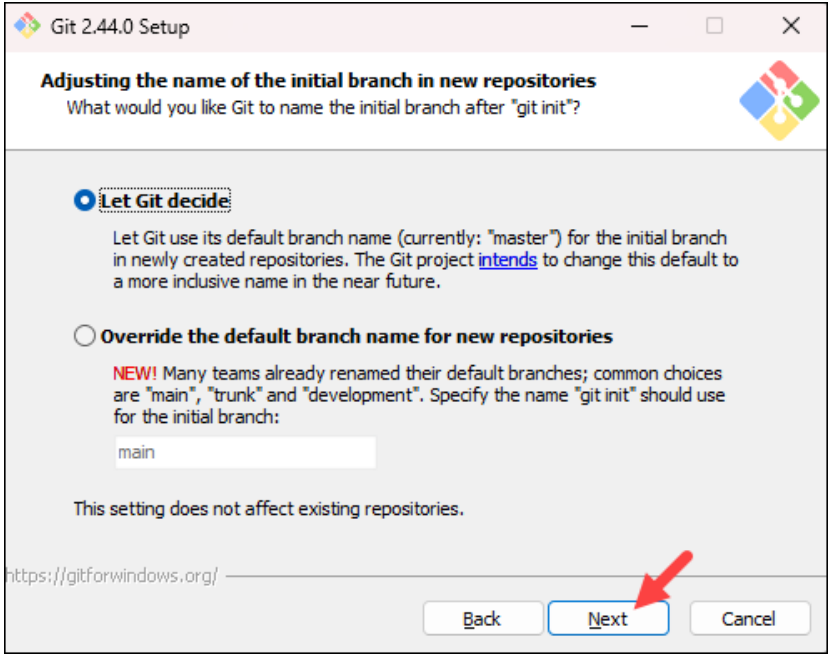
If you prefer to use a CLI text editor in **Git Bash**, select **nano** or **Vim** from the list.

Git's default text editor may not match your preferences. The default text editor on Linux and macOS is typically vim, and on Windows it's typically notepad



Adjusting name of initial branch

Allows you to choose a different name for your initial branch. The default is `master`. Unless you are working in a team that requires a different name, leave the default option and click Next.

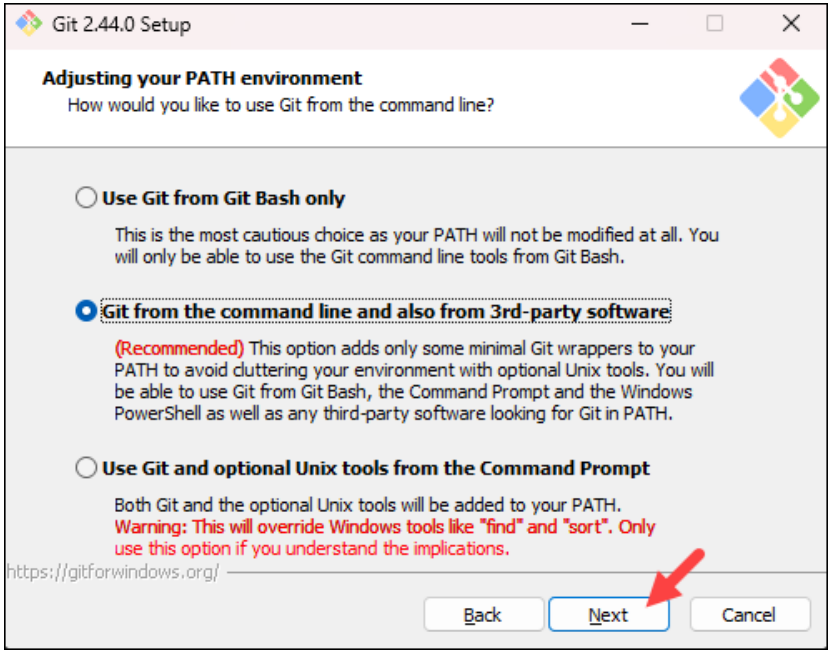


Adjusting path environment

The next step allows you to change the PATH environment. The PATH is the default set of **directories** included when you run a command from the command line. Keep the middle (recommended) selection and click Next.

Select the “Run Git from the Windows Command Prompt” option. Choosing this option will make it easy for you to run Git commands from the Windows Command Prompt (command line) if you choose.

The Git Bash tool works in the same way as the default Windows’ Command Prompt, but has some special features. With Git Bash you’ll be able to use a number of UNIX command line tools along with access to Git, and we recommend it since it’s often simpler to use than the Windows Command Prompt.



Choosing the SSH Executable

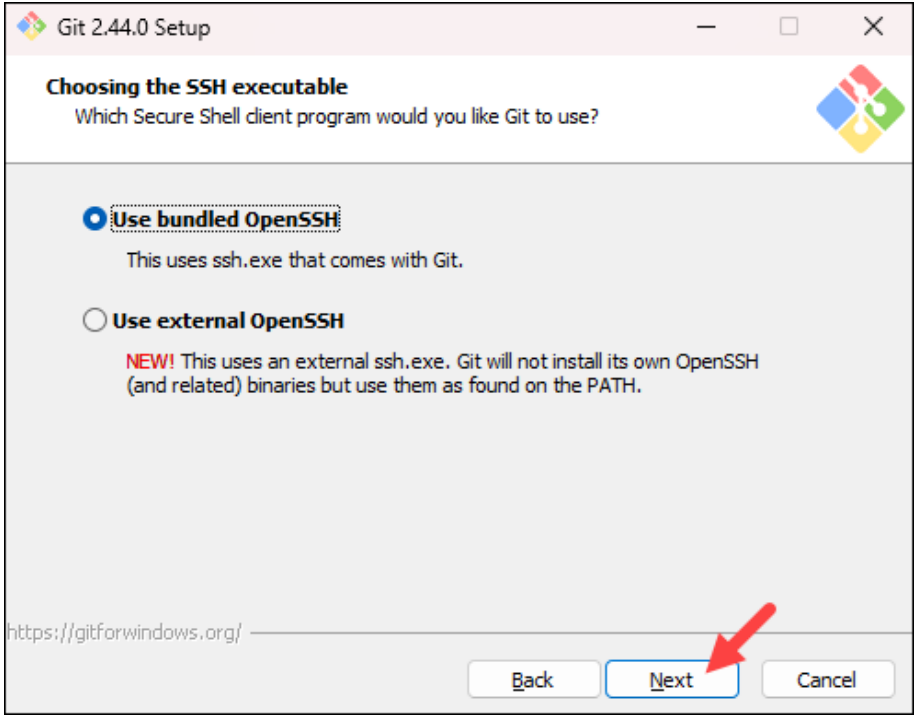
The installer prompts you to select the SSH client for Git to use. Git already comes with its own SSH client, so if you don't need a specific one, leave the default option and click Next.

Bundled OpenSSH
Uses the ssh.exe that comes with Git. Git will not install its own OpenSSH binaries, but will use them as found on the PATH.

External OpenSSH
Git doesn't install its own OpenSSH, so you can use other SSHs, like Windows OpenSSH.

When deciding between bundled and external OpenSSH, you can consider:

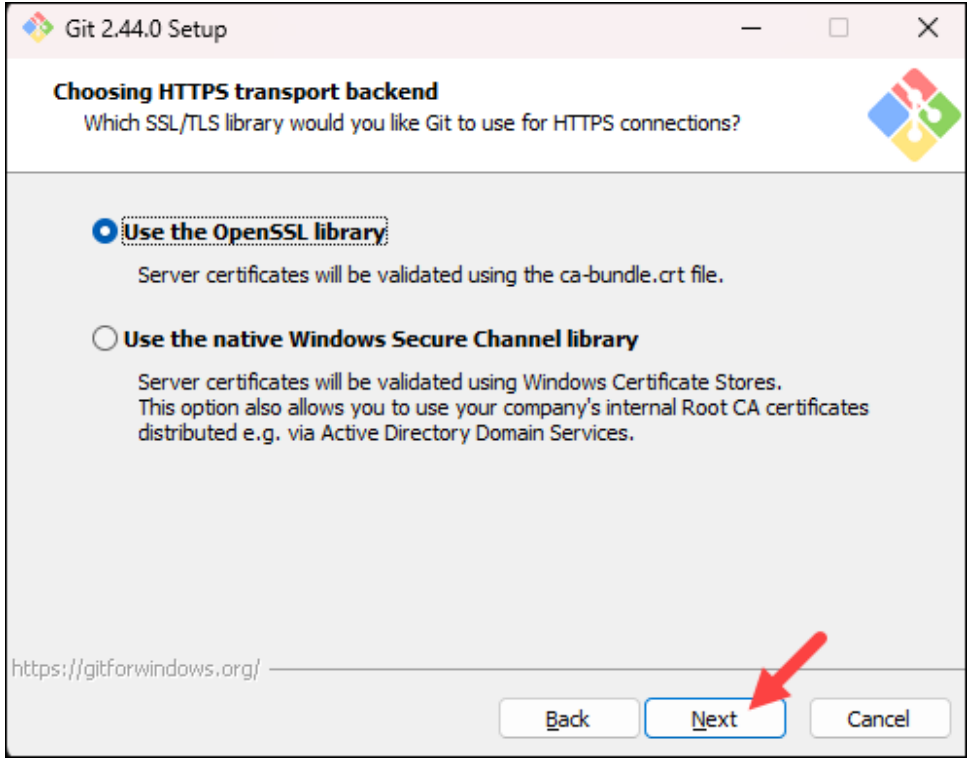
- **Bundled OpenSSH binaries**: A good option if you want to select what best fits your needs.
- **External OpenSSH**: Might be feasible for PuTTY users, but they usually involve more setup with IDEs and other tools.



Server certifications

The next option relates to server certificates. The default option is recommended for most users. If you work in an Active Directory environment, you may need to switch to Windows Store certificates. Select your preferred option and click Next.

If you are using git in an organization with enterprise-managed certificates, then you will need to use Secure Channel. If you are using git only to access public repositories (like GitHub) or your organization does not manage its own certificates, then you are perfectly fine using either SSL backend (they are just different implementations of the same protocol).

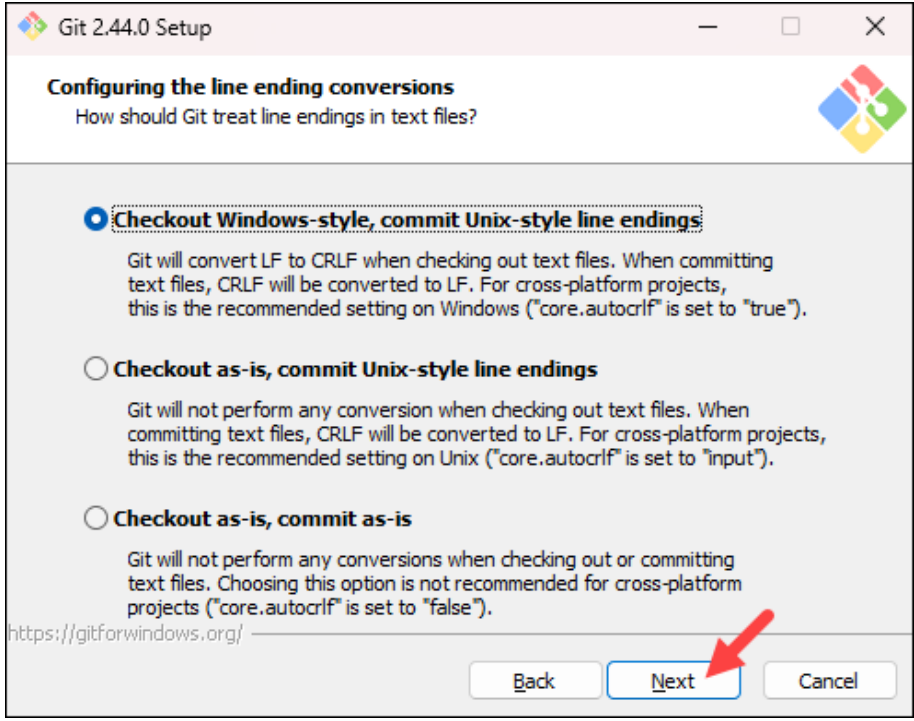


Configuring line ending conversions

This configures line-ending conversion, which relates to the way data is formatted. The default selection is recommended for Windows. Click Next to proceed.

Windows software uses two characters to indicate the end of a line in a text file (CRLF) while Unix and Linux systems use only a single character (LF).

Make sure to leave the option “Checkout Windows-style, commit Unix-style line endings” selected. This option makes sure that Git converts LF to CRLF when checking out text files. When committing text files, CRLF will also be converted to LF. This is a compatibility measure to protect newlines in text files, allowing you to easily work with text files on Windows and on Unix-style platforms.

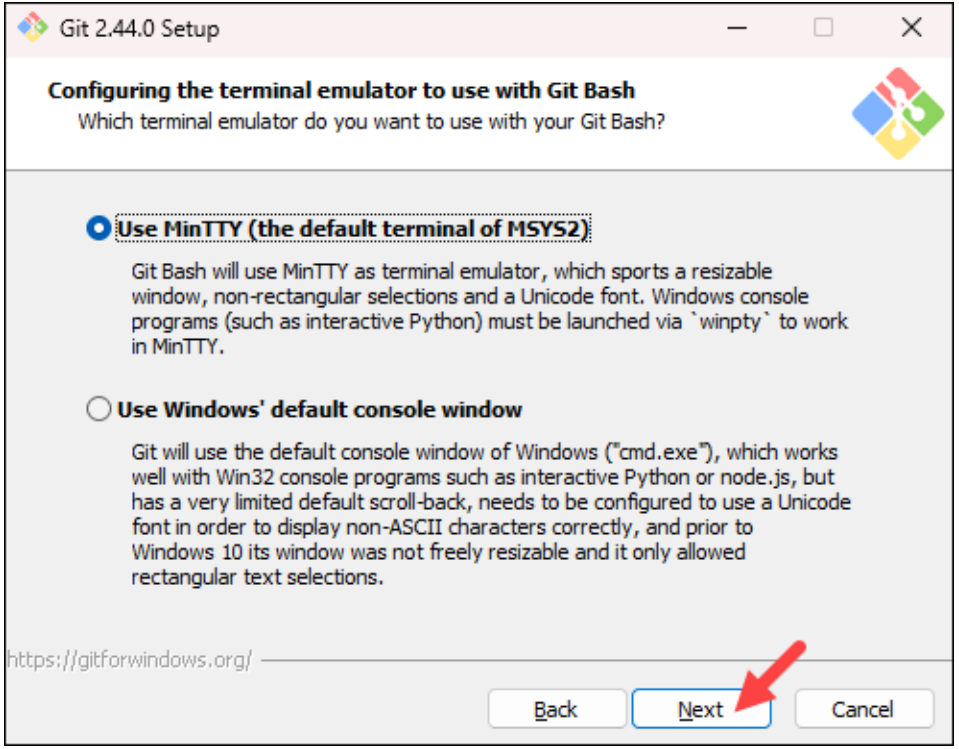


Terminal emulator

If you don’t use Git Bash at all, just ignore it and click next to go to the next step.

If you are familiar with Linux command line, then the first option is right for you (note: you can still run windows command while using MinTTY), otherwise, choose the second one.

As described in the wizard, the Windows console has some limitations: width is fixed (prior to windows 10), the scroll-back is very limited means that very limited command history, need to be configured to use Non-ASCII Character, etc… while MinTTY doesn’t.

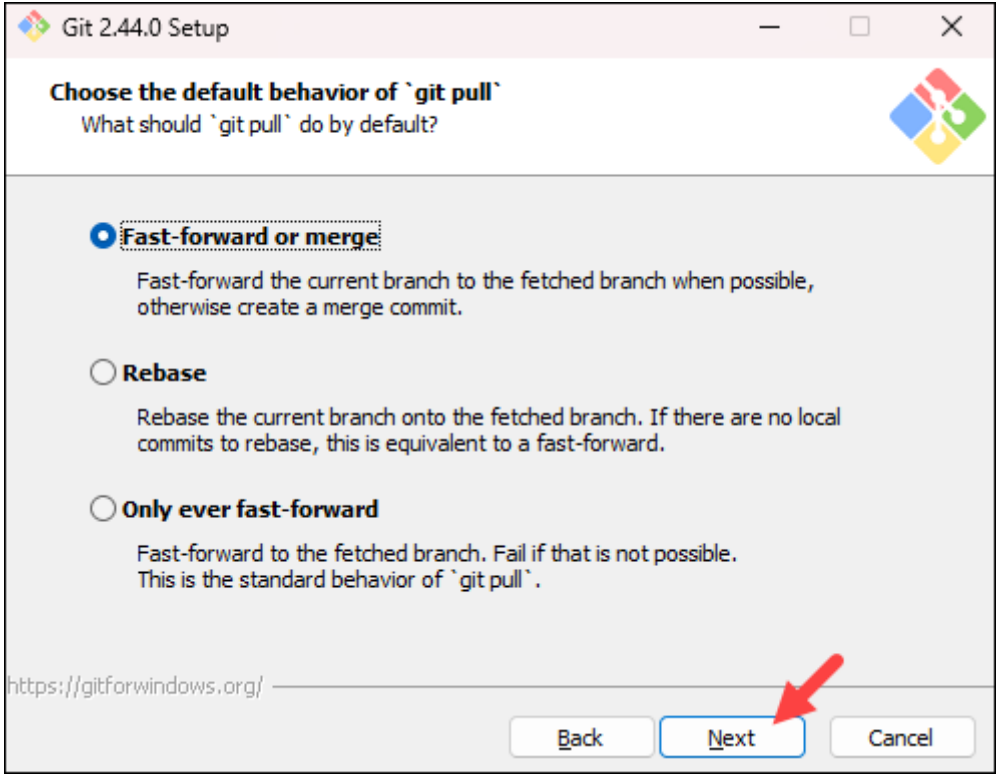


Choosing what git pull will do

The git pull command is actually a combination of two other commands, git fetch followed by git merge.

Rebase is done when you have branches outdated compared to master.Rebasing is a process to move a series of commits to a new base commit. It's useful for cleaning up your project history before merging changes.

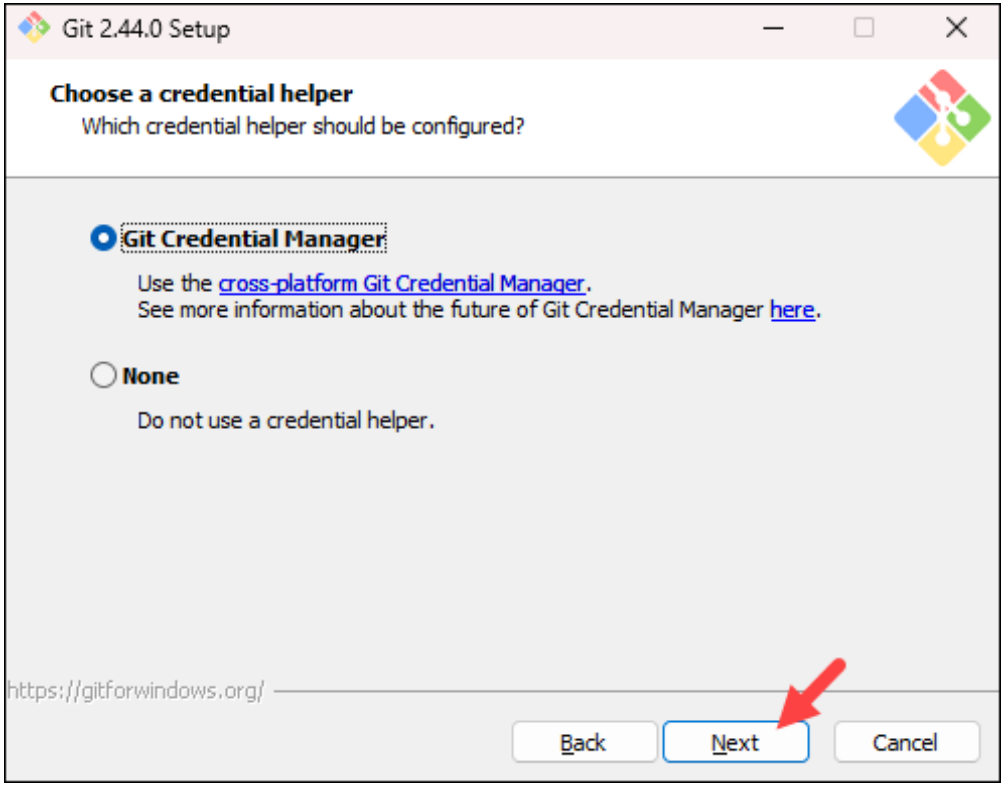
Choose default option to let git decide what is best on scenario basis.



Choose credential helper

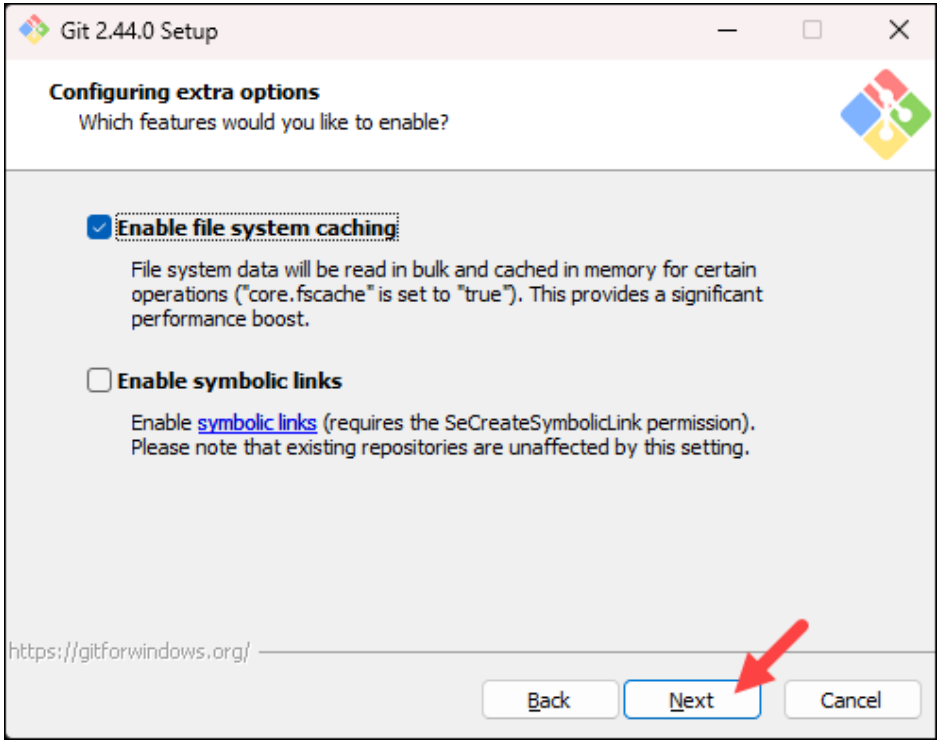
The next step is to choose which credential helper to use. Git uses credential helpers to fetch or save credentials. The default option is the most stable one.

The Git Credential Manager for Windows (GCM) provides secure Git credential storage for Windows. It's the successor to the Windows Credential Store for Git (git-credential-winstore), which is no longer maintained. Compared to Git's built-in credential storage for Windows (wincred), which provides single-factor authentication support working on any HTTP enabled Git repository, GCM provides multi-factor authentication support for Azure DevOps, Team Foundation Server, GitHub, and Bitbucket.



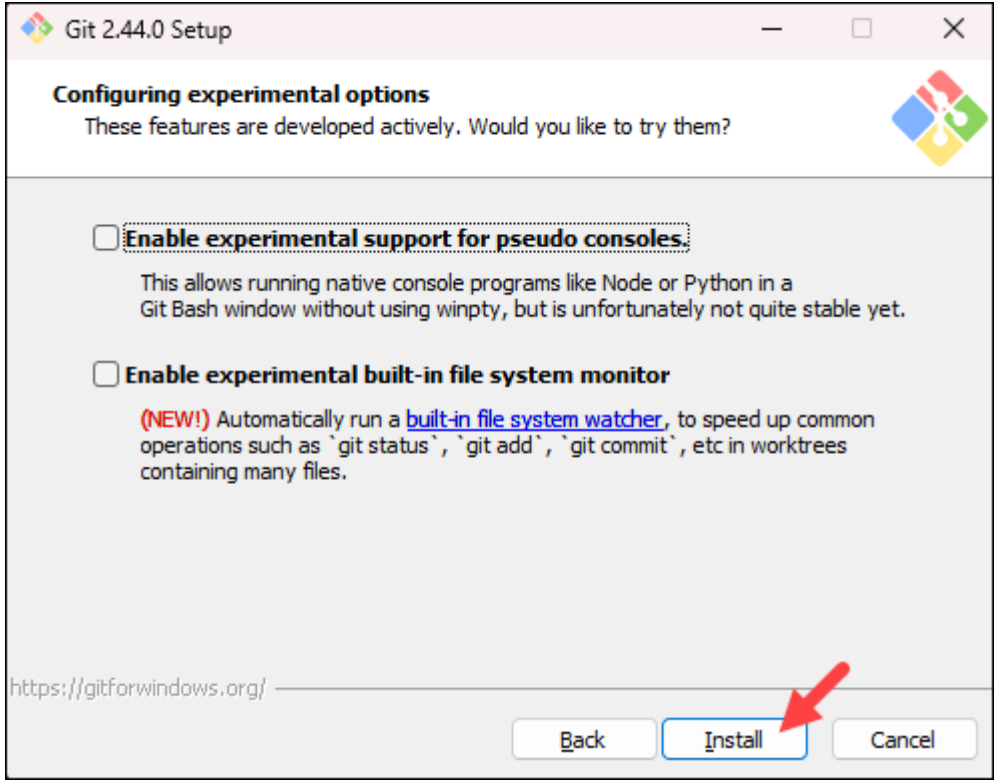
Configuring extra options

The next step lets you decide which extra options to enable. If you use **symbolic links**, which represent shortcuts for the command line, tick the box. Keep **file system** caching checked and click Next



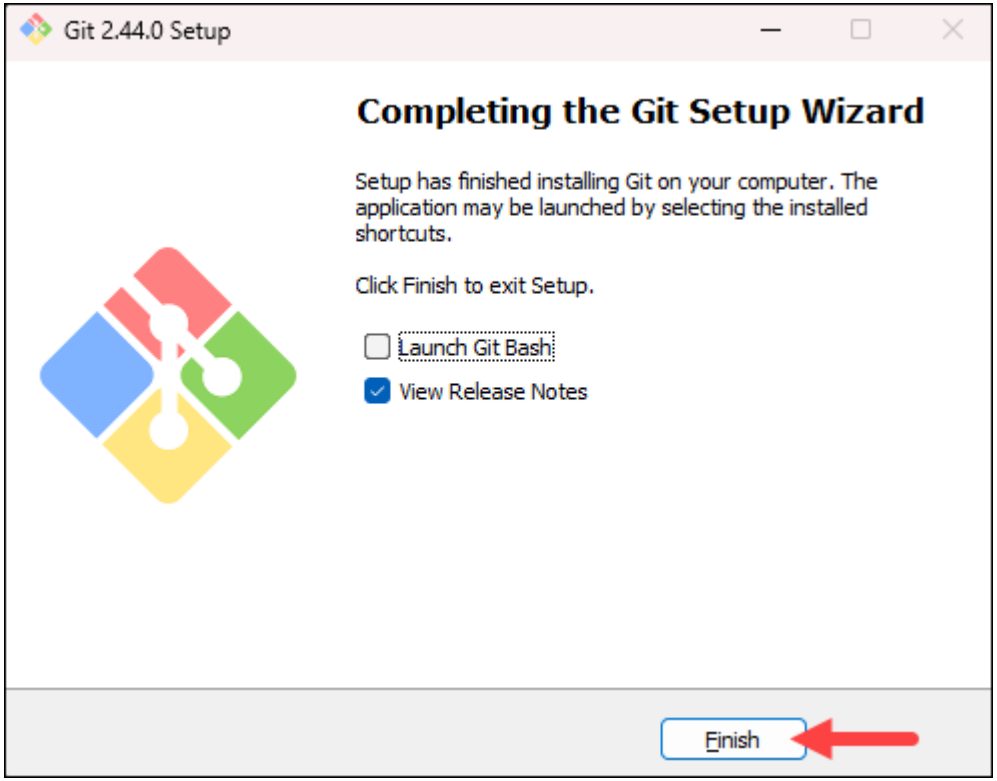
Configuring experimental options

Depending on which Git version you are installing, it may offer to install experimental features. At the time this article was written, the installer offered options to include support for pseudo controls and a built-in file system monitor. For the most stable operation, do not install experimental features and click Install.



Finishing

Once the installation is complete, tick the boxes to view the Release Notes or launch Git Bash if you want to start using Git right away, and click Finish.



Installing git on mac

Prerequisites

- A Mac machine.
- A user account with **root privileges**.
- Access to the terminal.

Installing GIT on MAC can be done through git installer (Open a **browser** and download the **Git installer** from Sourceforge) or homebrew (Recommended) or mac ports

Install Git on Mac Using Homebrew

To install homebrew run this code: curl -fsSL -o install.sh https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh

Install Git on Mac Using Homebrew

Use **Homebrew**, the package management system for macOS. Run the following `brew` command in the terminal to install Git:

```
brew install git
```

```
marko@Markos-Mac ~ % brew install git
==> Installing git
==> Pouring git--2.37.1.arm64_monterey.bottle.tar.gz
==> Summary
🍺  /opt/homebrew/Cellar/git/2.37.1: 1,559 files, 45MB
==> Running `brew cleanup git`...
Disable this behaviour by setting HOMEBREW_NO_INSTALL_CLEANUP.
Hide these hints with HOMEBREW_NO_ENV_HINTS (see `man brew`).
==> Caveats
==> git
The Tcl/Tk GUIs (e.g. gitk, git-gui) are now in the `git-gui` formula.
Subversion interoperability (git-svn) is now in the `git-svn` formula.

zsh completions and functions have been installed to:
  /opt/homebrew/share/zsh/site-functions

Emacs Lisp files have been installed to:
  /opt/homebrew/share/emacs/site-lisp/git
marko@Markos-Mac ~ %
```

Finishing Installation

Check the Git version to verify the installation:

```
git --version
```

```
marko@Markos-Mac ~ % git --version
git version 2.32.1 (Apple Git-133)
marko@Markos-Mac ~ %
```